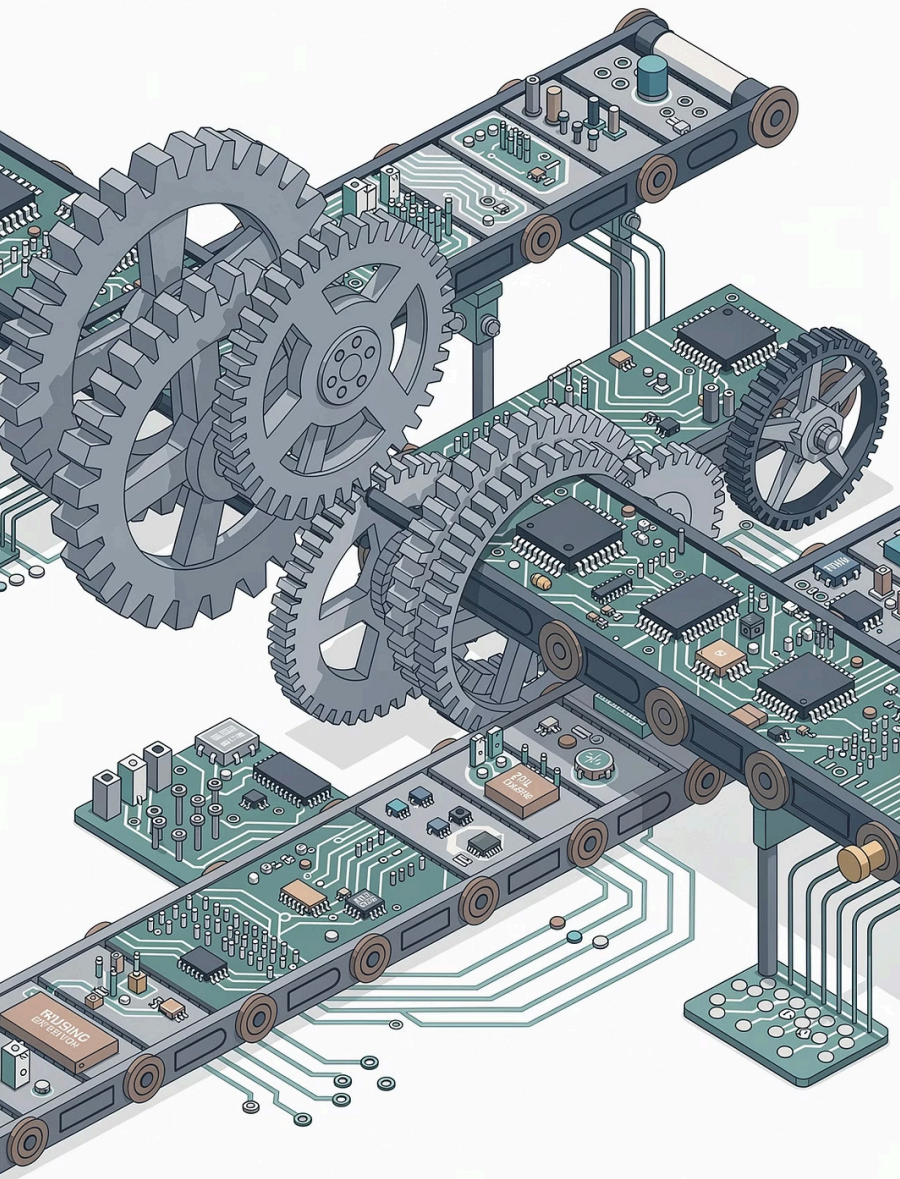




BuildForge: Dependency Build Engine Hackathon

Directed Graphs, Topological Sort & Implicit Graphs · Problem Statements

DEVELOPED BY TALENCIAGLOBAL

About This Hackathon

What is BuildForge?

BuildForge is a dependency build engine modelled on directed graphs. Each module is a vertex and a directed edge $u \rightarrow v$ means u must be built before v . Across six levels you will map the dependency graph, search an implicit state space, analyse the impact of a change, compute a valid build order, detect circular dependencies, and find the critical path that determines the minimum number of build stages.

Input Format & Rules

- Directed graphs given as N M followed by M directed edges u v (vertices $0..N-1$)
- Some levels then read a source vertex
- The implicit-graph level reads three integers: $LIMIT$ S T
- Adjacency lists are sorted
- Topological sort uses a min-heap for the lexicographically smallest valid order
- Levels run from intermediate to expert — no starter code provided

Levels Overview

Six progressive challenges taking you from graph fundamentals to expert-level dynamic programming on DAGs.

1	Map Dependencies ★★★ Intermediate Sources & Sinks in a directed graph
2	Deploy Counter ★★★ Intermediate Implicit graph state-space BFS
3	Impact Analysis ★★★ Intermediate Directed reachability via BFS
4	Build Order ★★★★ Advanced Topological sort with Kahn's algorithm
5	Circular Check ★★★★ Advanced Directed cycle detection via DFS colours
6	Critical Path ★★★★★ Expert Topological DP — longest chain & parallel width

Level 1 — Map Dependencies

★★★ INTERMEDIATE

DIRECTED GRAPH BASICS · SOURCES & SINKS

Scenario

Start by mapping the dependency graph. A directed edge $u \rightarrow v$ means module u must be built before module v . Read the graph and report its shape: the number of modules and dependencies, the **sources** (nothing depends on them — in-degree 0) and the **sinks** (they depend on nothing further — out-degree 0).

Concepts Covered

- Directed adjacency lists
- In-degree and out-degree
- Sources (in-degree 0) and sinks (out-degree 0)
- Reading a directed edge list

Formula Guidance

$\text{out-degree}(u) = |\text{adj}[u]|$ · $\text{in-degree}(v) = \text{number of edges into } v$

$\text{sources} = \text{count of in-degree } 0$ · $\text{sinks} = \text{count of out-degree } 0$

Problem Statement

Given: A line $N\ M$, then M directed edges $u\ v$ (u before v)

Required output:

- The number of nodes
- The number of edges
- The number of sources (in-degree 0)
- The number of sinks (out-degree 0)

Constraints:

- The graph is directed; store each edge only at its tail u
- A source has in-degree 0; a sink has out-degree 0
- Vertices are labelled 0 to $N-1$

Sample Run

Input:

```
6 6
0 1 / 0 2 / 1 3 / 2 3 / 3 4 / 3 5
```

Output:

Nodes: 6

Edges: 6

Sources: 1

Sinks: 2

⚠ Add a directed edge only to the tail's list (and bump the head's in-degree); mirroring it as in an undirected graph corrupts every degree.

Thinking Skill: In a directed graph each edge is one-way, so degree splits into in-degree and out-degree — sources and sinks (the build's natural start and end points) are exactly the zero-in and zero-out vertices.

Level 2 — Deploy Counter

★★★ INTERMEDIATE

IMPLICIT GRAPH · STATE-SPACE BFS

Scenario

The deploy counter starts at one value and must reach a target using the operations $+1$, -1 and $\times 2$, staying within $[0, \text{LIMIT}]$. The states form an implicit graph — never stored, only generated on demand — so a breadth-first search finds the fewest operations.

Concepts Covered

- Implicit graphs (neighbours generated on the fly)
- State-space BFS
- Shortest number of operations
- Bounding the search space

Formula Guidance

BFS over states $0..\text{LIMIT}$; neighbours of x are $x+1$, $x-1$, $x*2$ (kept in range)

$\text{dist}[S] = 0$; $\text{answer} = \text{dist}[T]$ (or -1 if unreachable)

Problem Statement

Given: A single line: $\text{LIMIT } S \ T$ (search within $0..\text{LIMIT}$, from S to T)

Required output:

- The minimum number of operations to turn S into T
- Whether T is reachable (Yes or No)

Constraints:

- From a value x the moves are $x+1$, $x-1$ and $x*2$
- Every intermediate value must stay within $[0, \text{LIMIT}]$
- Each move costs one step; S is at step 0

Sample Run

Input: 15 3 11

Output:

Steps: 3

Reachable: Yes

⚠ **Common Pitfall:** Bound every generated state to $[0, \text{LIMIT}]$ and mark states visited as you enqueue them; an unbounded or unguarded search explodes or revisits states endlessly.

💡 **Thinking Skill:** An implicit graph is still a graph — BFS does not care whether the adjacency list exists in memory or is produced by a rule, so the shortest-path guarantee carries over to state-space search.

Level 3 — Impact Analysis

★★★ INTERMEDIATE

DIRECTED REACHABILITY · BFS

Scenario

When a module changes, everything that depends on it — directly or transitively — must be rebuilt. Follow the directed edges from the changed module to find everything downstream: report the BFS visiting order and how many modules are affected.

Concepts Covered

- Directed reachability
- BFS following edge direction only
- Transitive dependents
- Counting the reachable set

Formula Guidance

BFS from S over the directed adjacency

reachable = number of vertices marked seen

Problem Statement

Given: A directed graph $N \times M$ with M edges, then a source vertex S

Required output:

- The BFS visiting order from S (following directed edges)
- The number of reachable modules (including S)

Constraints:

- Follow directed edges only in their direction
- BFS explores neighbours in sorted order
- Count S itself among the reachable modules

Sample Run

Input:
7 5
0 1 / 0 2 / 1 3 / 2 4 / 3 5
Source: 0

Output:
BFS Order: 0 1 2 3 4 5
Reachable: 6

⚠ Traverse edges only in their stored direction; following them backwards would pull in modules that depend on the change's dependencies rather than its dependents.

Thinking Skill: Reachability in a directed graph respects edge direction, so the impacted set is exactly the vertices BFS can reach from the change — upstream modules are deliberately left out.

Level 4 — Build Order

★★★★ ADVANCED

TOPOLOGICAL SORT · KAHN'S ALGORITHM

Scenario

Produce a build order that respects every dependency: each module appears only after all the modules it depends on. Kahn's algorithm repeatedly takes a module with no remaining dependencies; using a min-heap to break ties yields the **lexicographically smallest** valid order, and a leftover means a cycle.

Concepts Covered

- Topological sorting
- Kahn's algorithm with in-degrees
- A min-heap for the lexicographically smallest order
- Detecting a cycle (not all vertices ordered)

Problem Statement

Given: A directed graph $N \times M$ with M edges

Required output:

- The lexicographically smallest topological order, or CYCLE if none exists
- Whether the graph is acyclic (Yes or No)

Constraints:

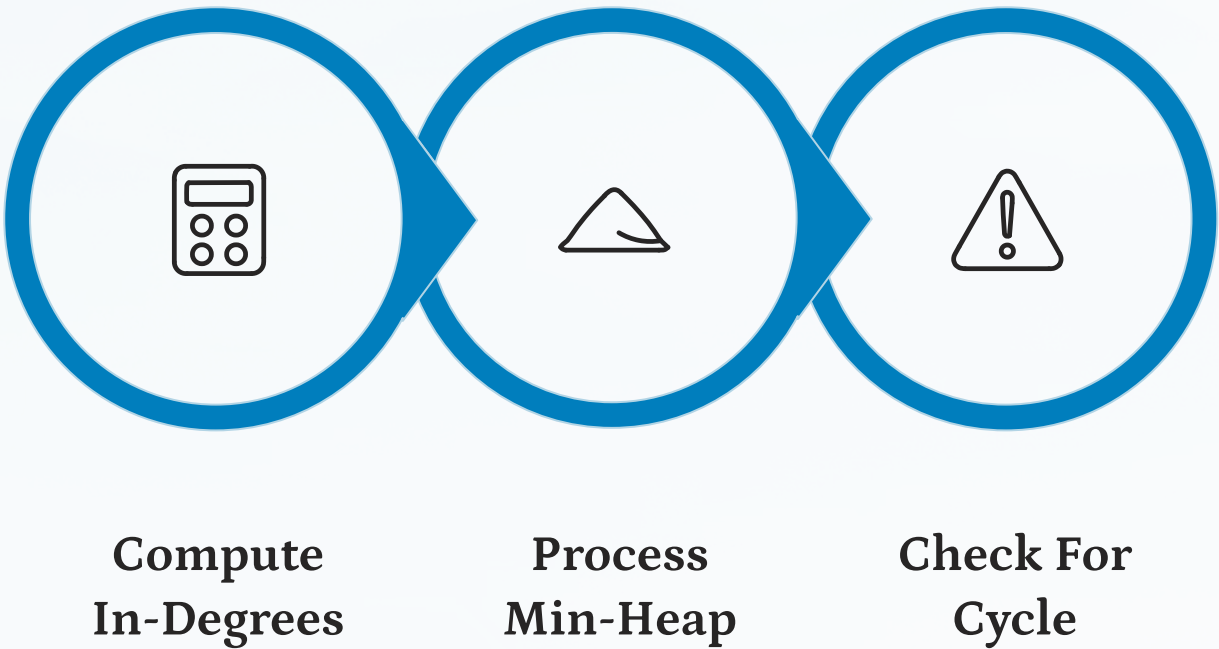
- A vertex may be output only after all its prerequisites
- Break ties by choosing the smallest available vertex (min-heap)
- If fewer than N vertices are output, the graph has a cycle

Sample Run

```
Input:
6 6
0 1 / 0 2 / 1 3 / 2 3 / 3 4 / 3 5

Output:
Order: 0 1 2 3 4 5
Acyclic: Yes
```

Formula Guidance



Common Pitfall: Use a min-heap (not a plain queue) for the lexicographically smallest order, and detect a cycle by checking that all N vertices were emitted — a cycle silently truncates the order.

Thinking Skill: Kahn's algorithm turns "respect all dependencies" into "repeatedly remove something with none left"; the min-heap makes the result canonical, and a stuck run is precisely a cycle.

Level 5 — Circular Check

★★★★ ADVANCED

DIRECTED CYCLE DETECTION · DFS COLOURS

Scenario

A circular dependency makes the project unbuildable. Detect one directly with a depth-first search that colours each vertex **white**, **grey** (on the current path) or **black** (finished): meeting a grey vertex closes a cycle. Also report how many modules are sources.

Concepts Covered

- Directed cycle detection
- Three-colour DFS (white / grey / black)
- The recursion stack as the "grey" set
- Distinguishing a back-edge from a forward/cross edge

Formula Guidance

colour: 0 = white, 1 = grey (on stack), 2 = black (done)

dfs(u): grey; for each successor w: if grey → cycle; if white recurse; then black

sources = count of in-degree 0

Problem Statement

Given: A directed graph $N \times M$ with M edges

Required output:

- Whether the graph contains a cycle (Yes or No)
- The number of sources (in-degree 0)

Constraints:

- A cycle exists when DFS reaches a grey (on-stack) vertex
- A finished (black) vertex is not a cycle
- Process disconnected graphs by restarting from each white vertex

Sample Run

Input:
5 5
0 1 / 1 2 / 2 1 / 0 3 / 3 4

Output:
Has Cycle: Yes
Sources: 1

 **Common Pitfall:** Only a GREY (on-stack) target signals a cycle; treating any already-visited vertex (including black) as a cycle gives false positives on a perfectly valid DAG.

 **White**

Unvisited vertex — not yet explored by DFS

 **Grey**

On the current recursion stack — a back-edge to grey = cycle!

 **Black**

Fully finished — all descendants explored, safe to ignore

 **Thinking Skill:** The grey set is just the current recursion stack; a directed cycle is exactly an edge back into that stack, which is why three-colour DFS detects cycles that a plain visited flag cannot.

Level 6 — Critical Path

★★★★★ EXPERT

TOPOLOGICAL DP · LONGEST CHAIN & PARALLEL WIDTH · CAPSTONE

Scenario

Close the engine by scheduling the build. The minimum number of sequential stages equals the longest dependency chain (the **critical path**); the most modules that can build in parallel is the largest number sharing a stage. Compute both with a topological-order dynamic program, and report the sinks.

Concepts Covered

- Longest path in a DAG via topological DP
- Assigning each vertex a stage (level)
- Maximum width (vertices per stage)
- Composing topo order with a DP

Formula Guidance

Process vertices in topological order; level initialised to 1

For edge $u \rightarrow w$: $\text{level}[w] = \max(\text{level}[w], \text{level}[u] + 1)$

$\text{stages} = \max \text{level}$ · $\text{width} = \max \text{count of vertices at one level}$

Problem Statement

Given: A directed acyclic graph $N \times M$ with M edges

Required output:

- The number of build stages (length of the longest dependency chain, in modules)
- The maximum parallel width (most modules sharing a stage)
- The number of sinks (out-degree 0)

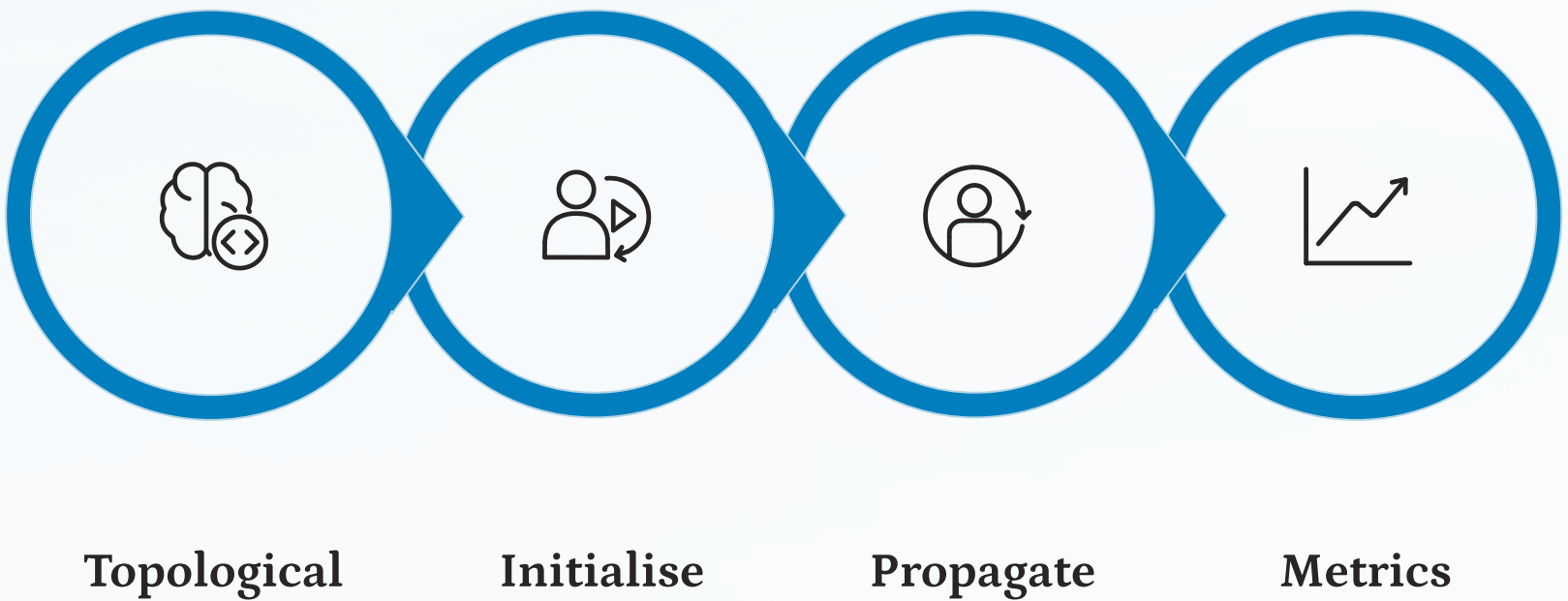
Constraints:

- Stage of a module = 1 + the maximum stage among its prerequisites
- Stages count modules along the longest chain (a single module is stage 1)
- Width is the largest count of modules assigned the same stage

Sample Run

Input:
6 6
0 1 / 0 2 / 1 3 / 2 3 / 3 4 / 3 5

Output:
Stages: 4
Width: 2
Sinks: 2



⚠ Common Pitfall: Relax along edges in topological order so each level is final before it is used; adding one per dependency step (not per edge visited blindly) is what makes the longest chain — and the stage count — correct.

💡 Thinking Skill: Processing vertices in topological order guarantees every prerequisite is finalised before its dependents, so a single forward pass computes the longest chain — the critical path that bounds how fast the build can possibly go.